

VEDA: Efficient LLM Generation Through Voting-based KV Cache Eviction and Dataflow-flexible Accelerator

Zhican Wang¹, Hongxiang Fan², Haroon Waris³, Gang Wang¹, Zhenyu Li¹, Jianfei Jiang¹, Yanan Sun¹, Guanghui He^{1*}

¹State Key Laboratory of Micro/Nano Engineering Science, School of Electronic Information and Electrical Engineering,

Shanghai Jiao Tong University, Shanghai, China,

²Imperial College London, United Kingdom, ³Institute of Space Technology, Pakistan

{wang_zhican, wangganginjstu, ambitious-lzy, jiangjianfei, sunyanan, guanghui.he}@sjtu.edu.cn
, hongxiangfan@imperial.ac.uk, haroon.waris@ist.edu.pk

Abstract—Large Language Models (LLMs) excel in natural language processing tasks but pose significant computational and memory challenges for edge deployment due to their intensive resource demands. This work addresses the efficiency of LLM inference by algorithm-hardware-dataflow tri-optimizations. We propose a novel *voting-based KV cache eviction* algorithm, balancing hardware efficiency and algorithm accuracy by adaptively identifying unimportant kv vectors. From a dataflow perspective, we introduce a *flexible-product* dataflow and a runtime reconfigurable PE array for matrix-vector multiplication. The proposed approach effectively handles the diverse dimensional requirements and solves the challenges of incrementally varying sequence lengths. Additionally, an *element-serial* scheduling scheme is proposed for nonlinear operations, such as softmax and layer normalization (layernorm). Results demonstrate a substantial reduction in latency, accompanied by a significant decrease in hardware complexity, from $O(N)$ to $O(1)$. The proposed solution is realized in a custom-designed accelerator, *VEDA*, which outperforms existing hardware platforms. This research represents a significant advancement in LLM inference on resource-constrained edge devices, facilitating real-time processing, enhancing data privacy, and enabling model customization.

Index Terms—LLM, Algorithm, Dataflow, Accelerator, Hardware

I. INTRODUCTION

Large language models (LLMs) [1], [15] have demonstrated remarkable performance across various natural language processing tasks, such as machine translation, text summarization, dialogue systems, and code generation. This potential has led to a growing trend of deploying private LLMs on edge devices (e.g., personal computers, smartphones, robotics, and intelligent vehicles), enabling real-time inference, data privacy, and model customization. However, the impressive performance of LLMs comes at the cost of significant computational effort and memory storage requirements, exacerbating the disparity between the substantial resource demands of LLMs and the limited resources available on edge devices.

The inference process of LLMs can be summarized in two phases: prefilling and generation. In the prefilling phase, input tokens are provided to the model in parallel, and the model encodes these tokens to kv vectors and concatenates them to form the KV cache. The primary operation in this phase is general-purpose matrix multiplication (GEMM), which is typically compute-bound. This process is similar to that of encoder-only Transformers, such as BERT and ViT [4], [5], with numerous dedicated accelerators being proposed for this task [6], [9], [16]. In the generation phase, tokens are generated sequentially in an auto-regressive manner. Each token attends to the entire KV cache formed in previous steps and extends the KV cache by adding the kv vector of the current token. Since only one token is generated at each step, the primary operation is general-purpose matrix-vector multiplication (GEMV), which is memory-bound. Very few solutions have been proposed to optimize the latter phase. Recent work [19] suggests batch scheduling to enable weight sharing across

multiple requests, which packages GEMV operations into GEMM for linear layers. While this is effective in cloud environments with multi-batch processing, edge devices typically use single-batch inference. Additionally, although batch scheduling improves performance for linear layers, it has limited impact on the attention process, as each user has a distinct KV cache, requiring the use of GEMV operations. Therefore, to enhance LLM inference efficiency on edge devices, particularly in the generation phase, this work proposes a comprehensive solution encompassing algorithmic improvements, dataflow optimization, and hardware architecture.

From the algorithmic perspective, numerous studies have shown that the attention process in LLM inference is highly sparse, with sparsity levels approaching 95% [21]. This means that only a small fraction of the KV cache is actively attended to, suggesting the feasibility of locating and maintaining only the pivotal positional kv vectors while discarding the others. Reducing the size of the KV cache not only decreases the computational effort required for critical modules of LLMs (e.g., the attention process) but also reduces off-chip memory access and memory consumption. However, existing algorithms have several limitations. **Some are simple to implement but suffer from accuracy loss.** For example, [18] proposes a sliding window strategy that retains only the earliest and most recent positional KV caches. While simple, it loses accuracy due to the inherent information loss from discarded out-of-window KV caches. [21] introduces a KV cache eviction strategy based on accumulated attention scores, but this approach has its own limitations, which we will discuss in this paper. **Other methods offer better accuracy but involve complex operations**, such as real-time top-k engines [16], complicated formula [2], which are inefficient for deployment. To achieve both hardware efficiency and algorithmic accuracy, we propose a novel *voting-based KV cache eviction* algorithm. Our approach is inspired by the voting process in democratic systems, where each token acts as a “voter” with a unique attention score. This score inherently reflects each token’s attention and relation to the kv vectors of prior tokens. Each token “votes” to determine which kv vector should be evicted, and by synthesizing these votes, we can efficiently remove unimportant kv vectors. This algorithm achieves superior accuracy compared to existing methods while maintaining hardware simplicity.

From the dataflow and hardware perspectives, we identify several challenges and propose corresponding solutions. **For GEMV operations:** First, widely ranged dimension size, the dimension size of matrix-vector multiplications varies significantly across different models and even within the same model. For example, in a $(1, k) \times (k, n) = (1, n)$ multiplication, the size of k can range from as small as 128 to as large as 11008 in Llama 7B. Second, dynamic inference process, in the generation phase, the KV cache increases incrementally with each step, meaning that the

This work was supported by the National Science Foundation of China under Grant 92464302. (*Corresponding author: Guanghui He).

dimensions of k or n change dynamically. However, existing GEMV accelerators, such as a multiplier array equipped with an adder tree, are designed with a fixed addition size, which leads to underutilization when k is smaller than the fixed size. For instance, suppose the multiplier array and the adder tree can perform 256—input multiplication and addition in parallel, but if the k is less than 256, the multiplier and adder tree are underutilized. Similarly, if k increases from 256 to 257, it increases one epoch of computation from 1 to 2, and the hardware is extremely underutilized for the second epoch. More importantly, the inability to utilize the sequence variation may make KV cache reduction less helpful to inference speed. Third, memory access irregularity, the transpose operation on the K matrix hinders inference due to irregular memory access patterns. Storing these vectors in the correct format offline is impractical because they are generated online rather than as static weights. While dedicated hardware modules could solve this issue, they would sacrifice efficiency. To mitigate these dataflow inefficiencies, we resort to mathematical interpretations for inspirations. Since GEMV can be interpreted as either inner-product or outer-product calculations mathematically, we propose *flexible-product dataflow* with a runtime reconfigurable GEMV PE array. This dataflow, combined with the dedicated PE array, efficiently handles size variations and eliminates the need for transpose operations, resulting in optimized inference.

For nonlinear operators, especially softmax and layernorm, we face several challenges. First, these operators introduce data dependencies that prevent parallel computation until all required data is available. Second, they occur among GEMV operations, and their latency can block subsequent GEMV computations. Third, the exponentiation and division operations required by these operators are more computationally expensive than multiplications and additions, and placing sufficient hardware resources for these operations can reduce latency but introduces significant overhead. To address these unprecedented challenges, we leverage the *flexible-product dataflow* and further propose a *element-serial scheduling* scheme. This approach not only resolves the latency bottleneck through parallel processing but also reduces the number of high-cost modules from $O(N)$ to $O(1)$ through architectural co-optimization.

Finally, integrating these comprehensive optimizations, we present *VEDA*, a dedicated LLM accelerator. To the best of our knowledge, this is one of the pioneer works to apply algorithm-hardware-dataflow tri-optimizations to accelerate LLM generation. Extensive results demonstrate the efficiency of our solution. In summary, we make the following contributions:

- We propose a novel *voting-based KV cache eviction* algorithm for LLM acceleration, achieving both efficiency and accuracy.
- We provide an insightful analysis of the challenges posed by GEMV and nonlinear operators in LLMs from both hardware and dataflow perspectives.
- We propose a *flexible-product dataflow* with a runtime reconfigurable PE array to address GEMV challenges and introduce *element-serial scheduling* to resolve the nonlinear operator bottleneck.
- We design *VEDA*, a dedicated LLM accelerator that efficiently supports both prefilling and decoding.

II. BACKGROUND

Figure 1 illustrates the structure of an LLM, with typical dimension sizes annotated. An input sequence, denoted as X_{in} , has a length of l tokens, each with a hidden dimension size of D , represented as (l, D) . The LLM computation consists of four main steps: Step 1 - QKV Generation: The input X_{in} is multiplied by

W_Q^i , W_K^i , and W_V^i to generate Q^i , K^i , and V^i for $i = 1, 2, \dots, H$, where H is the number of attention heads. Q^i , K^i , and V^i all have dimensions of (l, d) , with $D = Hd$, where d represents the head dimension. Step 2 - Attention Process: Within each head, Q^i is multiplied by K^{iT} to produce S^i of size (l, l) . Notice that due to the causality of LLM, each token only relates to the token before it, the upper triangle of S^i needs to be set $-\infty$. Each element of S^i is scaled to ensure numerical stability, followed by a row-wise softmax operation to generate the attention score matrix S'^i . S'^i is then multiplied by V^i , resulting in the output matrix O^i with dimensions (l, d) . Step 3 - Linear Projection: The outputs O^i from all heads are concatenated into a single matrix of size (l, D) , which is then multiplied by W_o . Step 4 - Feed-forward Layer: In this step, the input is multiplied by W_1 of size $(D, 4D)$ in FFN1, producing an intermediate result of size $(l, 4D)$, followed by a non-linear activation function like GELU or ReLU. The output is then multiplied by W_2 of size $(4D, D)$ in FFN2, after performing the layernorm (or RMSnorm), it produces the final output X_{next} .

LLM Inference Phase: LLM inference is divided into a prefilling phase and a generation phase. In the prefilling phase, l corresponds to the prompt length P . For example, in Figure 1, the prompt is “DAC is a” with $P = 3$. A key-value (KV) cache is constructed for each token during this phase, ultimately generating the output token “pioneer”, which marks the end of the prefilling phase. During the generation phase, l is always one, and activations are reduced to vectors. In the example, “pioneer” serves as input to the LLM, engaging in an attention process with previous KV caches and extending the KV cache with the current key-value vector. The output “conference” is then generated as the next token.

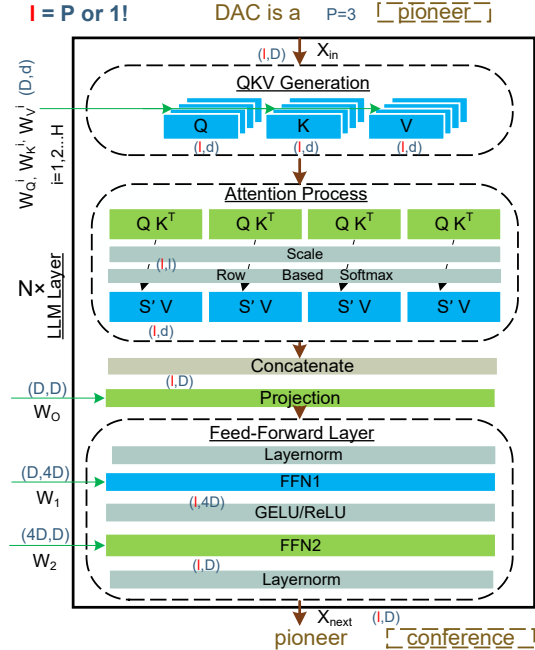


Fig. 1. Detailed LLM structure with sizes denoted. In this paper, matrix-form activations are represented in capital letters(e.g., KV cache), while lowercase letters denote vector-wise activations (e.g., attention score vector s'). Green and blue differentiate the dataflows, which collectively provide the optimal dataflow for LLM, which will be introduced in Section IV-B

III. VOTING-BASED KV CACHE EVICTION

Pivotal and Unimportant kv Vectors Identification. The key challenge in effective KV cache eviction is that we cannot pre-emptively determine which KV tensors are pivotal, and which are

unimportant at each new token generation step. Fortunately, some studies suggest that many pivotal kv vectors are likely to remain pivotal, and vice versa [8]. However, defining the importance of kv vectors is non-trivial, and accurately predicting the preferences of newly generated tokens remains challenging. Some research indicates that the most recent tokens are often pivotal, leading to the use of sliding window attention to maintain a fixed KV cache size. However, this approach suffers from accuracy degradation due to the loss of out-of-window tokens. Another commonly adopted method utilizes accumulated attention scores to define the importance of a kv vector. As illustrated in Figure 2 (a), the method accumulates the attention scores vertically across each column, generating an importance vector with a length matching the KV cache. Each element of the importance vector corresponds to the significance of each positional KV. The positional KV with the minimum value in the importance vector is selected for eviction, such as position 6 in the figure. However, several limitations have been identified:

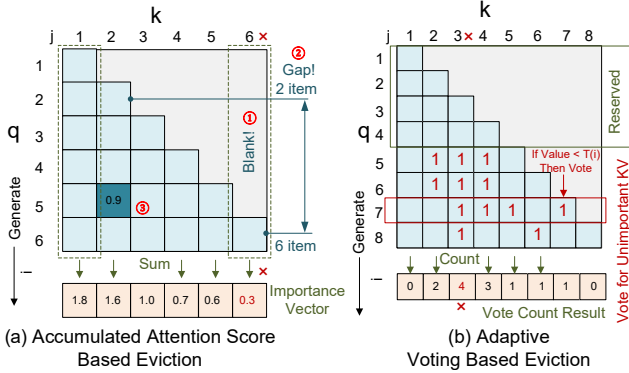


Fig. 2. A comparison between the influential accumulation-based eviction and our voting-based strategy.

Limitations of Existing Method. First, Item Count Bias, as indicated by the red ① in the figure. The number of items summed across different columns varies significantly, with 6 items for the first token, but only 1 for the 6_{th} token due to causal attention, where 5 elements are blank. This discrepancy leads to biased eviction, making recent KV tensors more likely to be pruned, which contradicts both intuition and studies suggesting that recent tokens are more important. Second, Criteria Bias, denoted by ② in the figure. When examining different rows, 2 items in the second row have a total attention score of 1, resulting in a mean of $\frac{1}{2}$, while 6 items in the 6_{th} row yield a mean of $\frac{1}{6}$. Hence, an attention score of $\frac{1}{3}$ may be considered insignificant in the second row but sufficiently important in the 6_{th}. Due to these varying criteria, simply summing values across rows may not yield optimal results. Third, Outlier Bias, indicated by ③. Some tokens have extremely high attention scores, indicating significance at certain positions but potentially less so at others. Nevertheless, the high scores are summed into the importance vector, leading to their consistent consideration as significant.

Overview. To address these challenges, we propose a *voting-based* strategy to differentiate pivotal and unimportant kv vectors. An overview is shown in Figure 2 (b). The method treats each generated token as a voter, fully leveraging the information in each attention score while adaptively adjusting the judgment criteria across positions. Specifically, alongside the attention score vector computation for each token, the algorithm also calculates an adaptive threshold for voting. If an attention score value is below this threshold, it is marked as unimportant, and a value of 1 is set in the corresponding position of the vote count vector. This process updates the vote count vector continuously, and when the cache reaches its predefined size,

the KV vector with the highest vote count is evicted. Additionally, a reserved length is maintained at the beginning, during which no voting occurs, aligning with the attention sink phenomenon [18].

Our evaluation shows that the proposed approach addresses the three above-mentioned limitations. First, it tends to preserve recent tokens; for example, in the step shown in the figure, position 7 of KV is voted for at most twice, whereas position 3 can be voted four times. Second, by applying an adaptive threshold relevant to each specific attention score vector, the criteria bias is effectively mitigated. Third, outlier bias is resolved by assigning a uniform weight of 1 instead of relying on absolute values.

Voting Eviction Algorithm

```

Set  $S = \text{Round}(r \cdot P)$ 
For ( $i = 0; i < L; i++$ )
     $s'(i) = \text{Softmax}(q(i) \times K^T)$ ;
    if ( $i < R$ )
        break; Reserved Stage
    else
        1 Mean =  $1/l(i)$ ;
        2 Compute  $\sigma$  of  $s'(i)$ ;
        3  $T(i) = a \cdot \text{Mean} - b \cdot \sigma$ ;
        4 Compute Vote( $i$ ) by  $T(i)$ ;
        5  $C(i) = C(i) + \text{Vote}(i)$ ;
        if ( $i \leq P$ )
            break;
        else
            Eviction Stage
             $j = \text{Index}(\text{Max } C(i))$ ;
            Evict KV( $j$ );

```

Notations

S : Targeted KV Cache Size
 r : Compression Ratio
 P : Prompt Length
 i : Index of Generating Token
 L : Total Generation Length
 R : Initial Reserved Length
 $s'(i)$: Attention Score
 a, b : Hyper-parameters
 $\text{Vote}(i)$: Voting Count Result
 $T(i)$: Threshold to Vote
 $C(i)$: Voting Count Result
 j : Index of KV to be Evicted

Fig. 3. Voting-based KV cache eviction algorithm with notations on the right.

Details. The complete algorithm is illustrated in Figure 3, with notations on the right, aligned with their order of occurrence. Common operations in LLM inference are omitted for clarity. The algorithm begins with an adjustable parameter r , provided by the user, representing the compression ratio, enabling a trade-off between aggressive compression and precision. The prompt length P is determined once the prompt is input into the model. Based on P and r , the target KV cache size S is calculated. This configuration works well for tasks such as text summarization and question answering, though a fixed target size can also be set for language modeling as the sequence length extends from 1 to the model's maximum capacity.

The algorithm operates through both the prefilling phase and the generation phase. The prefilling phase is divided into the reserved stage and the voting stage. Voting does not occur during the reserved stage, with its size R set to 32, establishing the lower bound for the KV cache. Line 3 of the voting stage defines our adaptive threshold, as a linear combination of the mean and standard deviation of the attention scores. A sparse attention score results in a higher standard deviation, leading to a lower threshold, while a more evenly distributed score has a smaller standard deviation, producing a higher threshold—thus dynamically addressing the criteria bias. The hyper-parameters a and b can be fine-tuned through model-specific calibration without affecting deployment efficiency; values of $a = 1$ and $b = 0.2$ generally prove effective. Notably, the threshold may theoretically drop below zero, in which case the algorithm identifies the minimum attention score and votes accordingly. After processing the entire prompt, the algorithm transitions to the generation phase, which includes voting and eviction stages. Voting continues for each current token, and the algorithm evicts the kv vector with the highest vote count. In cases where multiple positions have identical vote counts, the earliest position is selected for eviction. This strategy significantly accelerates the generation phase due to the reduced cache size.

IV. FLEXIBLE DATAFLOW OPTIMIZATION

A. Flexible-product Dataflow

Motivations. While dataflow optimization [3] and dataflow-optimized accelerator [17] for the LLM prefilling phase (or encoder-only Transformers) have been studied, a gap remains for the LLM generation phase. As mentioned in Section I, the wide range of dimension sizes, the dynamic inference process, and irregular memory access during transposition hinder efficiency. Notably, the current KV cache compression developed by the algorithm community significantly reduces storage overhead but is less effective in accelerating inference. Specifically, the GEMV of the attention process can be represented as $q \times K^T = s$ with the size $(1, d) \times (d, l) = (1, l)$ and $s' \times V = o$ with $(1, l) \times (l, d) = (1, d)$. With l varying due to token growth during the inference process or KV cache reduction, existing architectures lack the efficiency to handle such variability. To address this, we return to mathematical interpretation for inspiration. **Mathematical Definition:** There are two ways to

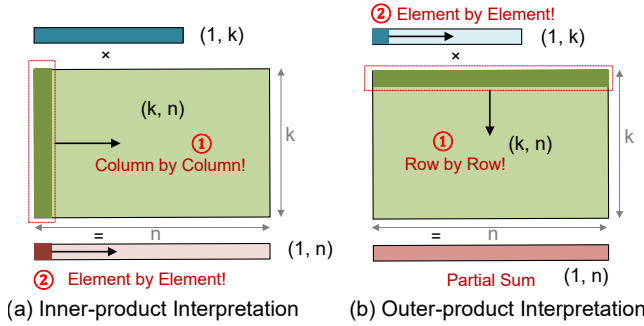


Fig. 4. Two mathematical interpretations of GEMV.

interpret GEMV, as shown in Figure 4. Given a general-purpose notation $(1, k) \times (k, n) = (1, n)$, for the inner-product method, the entire input vector $(1, k)$ is multiplied by the weight matrix (k, n) , column by column, generating the output vector $(1, n)$ incrementally. This interpretation is widely used, especially in adder-tree-based implementations. For the outer-product method, one element of the input vector is multiplied element-wise by each row of the weight matrix sequentially, generating a partial output vector $(1, n)$ of the same size as the final output. This partial output is accumulated into the result until all rows of the weight matrix are processed. **Flexible-product Inspiration:** For efficient memory access, denoted by ① in Figure 4, the weight memory access patterns of the two interpretations differ: one operates column-wise, while the other operates row-wise. Interestingly, it inspires us that storing the matrix uniformly, instead of performing transposition, a hybrid implementation on the computation engine can have the equivalent effect to the transpose operation. Specifically, in the attention process, the KV matrix can be uniformly stored in (l, d) format, where l denotes the address, and d elements share the same address. In each step, a $(1, d)$ vector of the K or V matrix can be accessed with a single memory operation, allowing the computation of $q \times K^T = s$ with $(1, d) \times (d, 1)$ segment of K^T per step, l steps to realize $(1, d) \times (d, l) = (1, l)$ in an inner-product approach. Similarly, to compute $s' \times V = o$, element-wise multiplication is performed between $(1, 1)$ and $(1, d)$ segment of V per step, l steps to realize $(1, l) \times (l, d) = (1, d)$, in an outer-product approach. This strategy avoids the costly transpose operation, ensuring a consistent storage format between the K and V matrices.

For efficient computation, denoted by ② in Figure 4, we observe that either the input or the output is accessed in an element-by-element manner, which offers flexibility in the corresponding

dimension, indicating that arbitrary size in that dimension can be temporarily mapped to computation cycles. Specifically, for $q \times K^T = s$ with size $(1, d) \times (d, l) = (1, l)$, let l correspond to n and d correspond to k in Figure 4 (a); the variation in sequence length l is effectively mapped to computation cycles. Similarly, for $s' \times V = o$ with $(1, l) \times (l, d) = (1, d)$, let l and d correspond to k and n , respectively in Figure 4 (b), l variations can be translated into computation cycles variation. In summary, analyzing the problem from both computational and storage perspectives led to a consistent conclusion: using the inner product for $q \times K^T$ and the outer product for $s' \times V$ is the optimal choice, leaving the primary challenge of designing a runtime-reconfigurable computation engine.

Runtime Reconfigurable PE Array: Reconfigurable PE. As illustrated in Figure 5 (a), each PE is governed by a 2-bit control signal, allowing it to perform local partial sum accumulation, transmit partial sums, clear registers, or disable. The multiplexers enable different modes to specify whether the partial sum is accumulated locally or received from other PEs. The dotted section, present only in type-B PEs, indicates that both adder inputs are sourced from other PEs. **Outer-Product Configured PE Array.** PEs are structured as shown in Figure 5 (b) to perform an outer-product operation. Each PE operates independently with distinct weights, without spatial partial sum transmission, while sharing a broadcast input scalar. The PE array is set as 8×8 , a typical finest dimension in modern LLMs. The structure and mapping of the PE are depicted on the right. For $s' \times V$, l is temporally mapped to ensure flexibility, meaning that in each cycle, one scalar from s' is broadcast to the PE array. The parameter d is spatially assigned to each PE, with local accumulation of the partial sum. **Inner-Product Configured PE Array.** In Figure 5 (c), weights and inputs are independently multiplied within each PE in parallel during each cycle, with partial sums transmitted across PEs. The PE adders are arranged into a two-level adder tree structure. The L1 adder tree receives multiplication results and performs accumulation at the row level, while the L2 adder tree aggregates the results from the L1 adder tree. For $q \times K^T$, l is mapped spatially, allowing one output element of s to be calculated per cycle, with the entire computation completed in l cycles. The parameter d is spatially mapped across the PE array. **Hierarchical Adder Tree Implementation.** As shown in Figure 5 (d), the red annotations indicate the type and location of each PE in the row. The adder tree is configured with PEs 1, 3, 5, 7 designated as type-A, where one input to the adder is the local multiplication result, and the second input is the multiplication output from PEs 2, 4, 6, 8, respectively. PEs 2, 4, 6, 8 are type-B, with both operands sourced from other PEs. The L2 adder tree follows similar design principles as the L1 adder trees, with the 8_{th} PE functioning as the local adder within a PE, compared to the L1 adder tree.

B. Element-serial Scheduling

Motivations. Nonlinear operations such as softmax and layernorm impose significant data dependencies, as computation cannot proceed until all relevant elements are mathematically ready. These data dependencies impede parallel processing, and the inherent latency delays subsequent operations. As shown in Figure 6, conventional scheduling causes the PE array to remain idle during the computation of softmax or layernorm in the Special Function Unit (SFU). Although increasing the computation units can reduce latency, it leads to higher overhead. Previous research [14], [20] suggests using approximate units to mitigate costs. In contrast, we innovatively address the issue by co-designing dataflow scheduling and architecture, aiming to minimize the number of SFUs while eliminating latency, as illustrated in Figure 6 (a). **Element-serial**

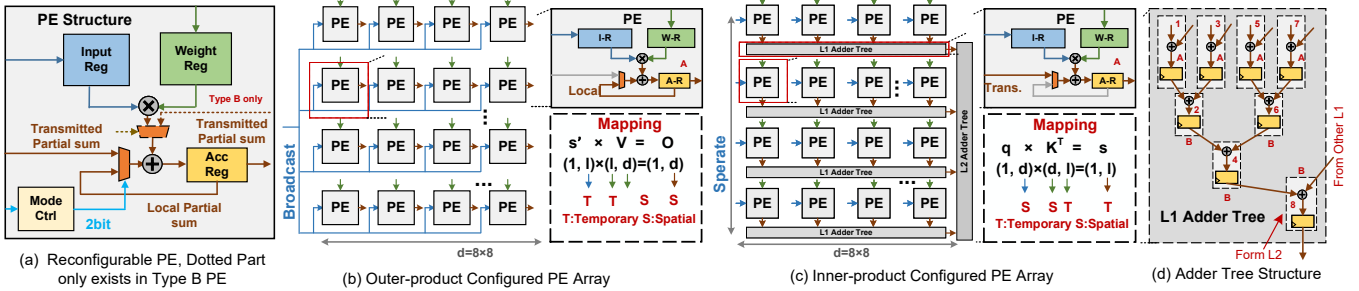


Fig. 5. Runtime reconfigurable hardware architecture.

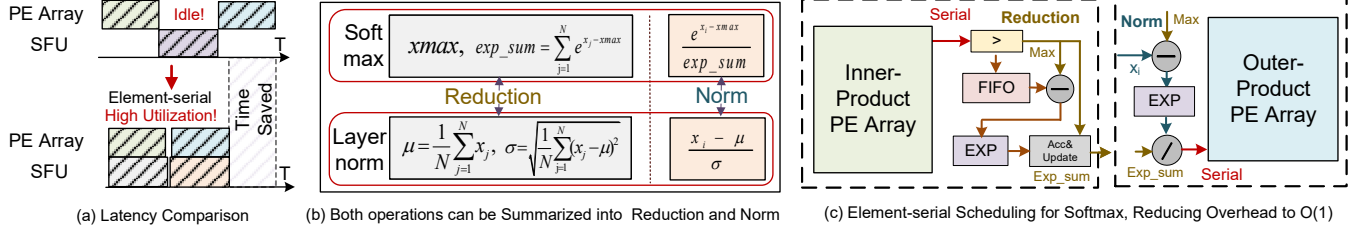


Fig. 6. Element-serial scheduling.

Scheduling. We summarize both softmax and layernorm operations into two stages: reduction and normalization (norm). In the reduction stage, elements are condensed into two numerical characteristics: maximum and exponent summation (exp_sum) for softmax, and mean and standard deviation for layernorm. The normalization stage involves element-wise computations—subtraction, exponentiation, and division for softmax; subtraction and division for layernorm. Drawing from the inspiration of the element-by-element manner discussed in Section IV-A, we propose to schedule the reduction stage element-by-element, running in parallel with PE array computations, which performs operations on the serial output of an inner-product-configured PE array; and scheduling the normalization on the serial input of an outer-product-configured PE array, concurrently with PE array computations as well. A specific example of softmax is demonstrated in Figure 6 (c). During the computation of $q \times K = s$ using the inner product in the PE array, the reduction unit in the SFU receives the serial, element-by-element output. It then identifies the maximum value for each tile, updates the maximum, and stores the tile in a FIFO. Once a tile is stored, its maximum is obtained, and this current maximum is used to update the exp_sum , the process is similar to [10]. When the computation of all l elements is completed, the final exp_sum is determined in just a few cycles. For $s' \times V = o$, continuing with the inner product would costly require the concurrent preparation of $(1, l)$ softmax normalization results. Therefore, the outer product configuration facilitates element-by-element serial input into the PE array, enabling parallel processing in SFU. Notably, with a PE array of size d , a single SFU suffices to remove latency in *element-serial scheduling* as shown in Figure 6 (a), and reduce the SFU cost from $O(N)$ to $O(1)$. The same technique applies to layernorm: inner-

product implemented GEMV before reduction, the sum of element and the sum of element square is computed simultaneously through the element-serial output to obtain mean and variance; and outer-product implemented GEMV after normalization, the subtraction, and division are performed through element-serial input. Since both softmax and layernorm are positioned between GEMV operations, they effectively define the optimal dataflow for the entire LLM: Please refer to Figure 1, where green indicates the inner product and blue represents the outer product.

V. HARDWARE ARCHITECTURE

Combining dataflow, hardware, and algorithm optimizations, we designed VEDA, as depicted in Figure 7. The architecture comprises a PE array, Special Function Unit, voting engine, scheduler, and HBM serving as the off-chip memory. Detailed hardware specifications are provided in Table I. VEDA supports both prefilling and generation phases. For storage, weights are fetched to the on-chip buffer and reused across tokens to minimize off-chip memory access for the prefilling phase; however, during the generation phase, they are directly input into the PE array. Thanks to the effective transposition elimination through flexible dataflow, the KV cache is uniformly stored in (l, d) , ensuring the high utilization of HBM bandwidth, they need to be stored off-chip, and they can be regarded as weight for the PE array during the generation phase. Additionally, the vote count also needs to be stored and accessed from off-chip to reduce the on-chip overhead. For computation, the GEMM of the prefilling phase is achieved through multiple GEMVs, and weight is reused across GEMVs through buffering. Another significant benefit of GEMV implementation is that, compared with GEMM kernel computation, the effective operations of the attention process are reduced to half due to the causality, our flexible PE array can directly skip computing the upper triangle half of the attention score.

Since the primary modules have been introduced, we will now focus on the voting engine's mechanism. It receives the softmax result s' (which also serves as input for $s'V$) and stores it in the FIFO. Simultaneously, s' is internally input into the reduction unit to compute the mean and standard deviation. Once stored in the FIFO, the reduction unit also finishes the computation of mean and standard deviation, providing a threshold. Each element is then sequentially output from the FIFO to determine voting eligibility, update the previous vote counts, and adjust the registered vote count (hardware

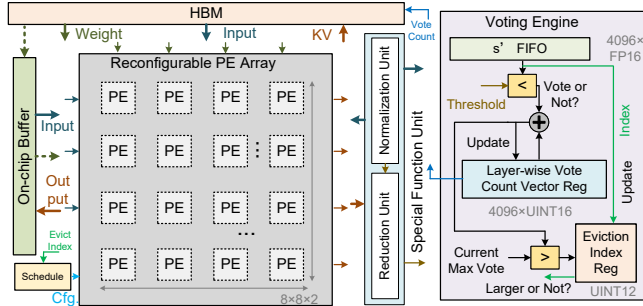


Fig. 7. Hardware architecture of VEDA, with blue, green, and red denote input weight and output.

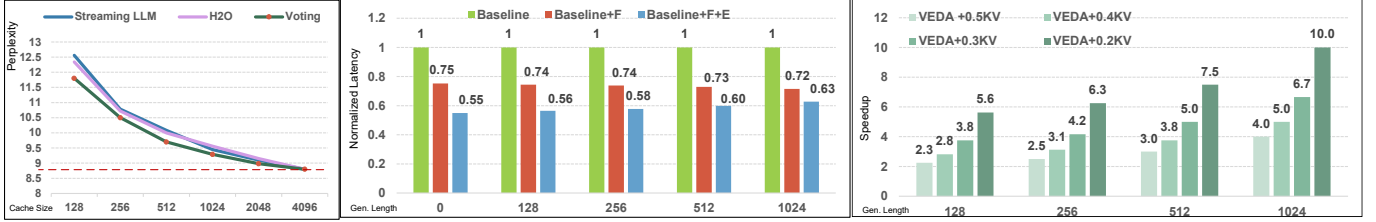


Fig. 8. Experiments. Left: Language modeling evaluation. Center: Ablation study of dataflow optimization. Right: Speedup by voting-based cache eviction.

implemented by buffer) accordingly. During the prefilling phase, no eviction is performed. However, in the generation phase, the vote count is compared with the current maximum vote to update the eviction index. Voting operates layer-wise, meaning that all heads are aggregated and averaged. Once s' across all heads is processed, the final eviction index is established, indicating that a particular address of kv vectors will no longer be accessed. The voting engine consistently operates in parallel, ensuring it does not affect the computations of other modules.

VI. EVALUATION

Experiment Setup. **Hardware:** We utilize SystemVerilog to implement the VEDA architecture, using FP16 as the default arithmetic format. The design is synthesized using the Design Compiler under the TSMC 28nm CMOS library with 1GHz to evaluate area and power consumption. A 256GB/s HBM is employed through Ramulator [7] to model the off-chip memory. For the on-chip buffer and FIFO, CACTI [11] is used to estimate energy consumption and area. A cycle-accurate performance model is employed to measure latency, which is cross-validated with RTL simulations. The hardware metrics are detailed in Table I, demonstrating that PE and buffer dominate the area and power, SFU consumes less than 3% due to *element-serial scheduling*, and the voting engine incurs a small 6.5% of overhead. **Software:** We choose the influential Llama-2 7B model, with a maximum sequence length of 4096, and use the language modeling task—a standard benchmark for assessing LLM performance—to validate our algorithm’s feasibility. The primary evaluation metric is perplexity, with lower values indicating better performance.

TABLE I
HARDWARE DETAILS, WITH AREA AND POWER BREAKDOWN OF VEDA

Modules	Parameters	Area/ mm^2	Power/ mW
PE Array	8*8*2 Reconfigurable PEs	0.493	175.64
Voting Engine	4096*16bit FIFO 4096*16bit Vote Buffer & Others	0.069	26.41
Special Function Unit	2 EXP, 2Divider, 1Sqrt & 2Multiplier and 4Adder 32*16bit FIFO	0.029	13.19
Schedule	System Control & PE Array Config	0.041	11.20
On-chip Buffer	256KB SRAM	0.426	148.82
Off-chip DRAM	HBM, Bandwidth=256GB/s		
Total	TSMC 28nm: Area=1.058 mm^2 Power = 375.26 mW		

Algorithm Evaluation: We report perplexity results using 1000 samples from the PG-19 dataset [12], a widely-used benchmark for LLM language modeling evaluation. We compare our *voting-based* cache eviction strategy with the notable Streaming LLM [18] and H2O [21] across various cache sizes. As depicted in Figure 8 (left), the *voting-based* eviction consistently outperforms both H2O and Streaming LLM, owing to the voting mechanism’s ability to adaptively eliminate three types of bias and select the optimal key-value vector for eviction. Furthermore, even when the KV cache is reduced to 0.1 of the original length (4096), the accuracy loss remains negligible, demonstrating the feasibility of our algorithm.

Ablation Study. For dataflow and reconfigurable hardware gain, to assess the impact of our flexible architecture (F) and *element-serial scheduling* (E), we use a conventional adder-tree-based architecture as the baseline, similar to A^3 [6] Transformer accelerator, in which the softmax operation functions as a pipeline stage with inherent latency. The conventional adder-tree-based design is constrained to a fixed dataflow for inner product computations, lacking flexibility. To ensure a fair comparison, accelerators are configured to have identical peak throughput and the same number of SFUs as VEDA. We employ the Llama-2 7B model with a prompt length of 512 for the prefilling phase and generate sequences ranging from 0 to 1024 tokens in the generation phase. The latency of the attention process is averaged over tokens during the generation phase, and depicted in Figure 8 (center), the *flexible-product* approach consistently achieves over a 25% latency reduction due to higher utilization across varying lengths. The *element-serial scheduling* further reduces latency to approximately 60%. For algorithmic gain, we use the same 512-length input and vary the maximum generated sequence length between 128 and 1024, averaging the latency of the attention process over tokens. The baseline in this evaluation is VEDA without cache eviction. The *voting-based* approach effectively maintains a constant KV length ($512 \times \text{ratio}$) throughout the generation phase, in contrast, the baseline’s KV cache grows with each step. This approach achieves a 2.3 ~ 10.0 $\times$ speedup over the baseline.

Comparison. Table II shows that, compared with SOTA accelerators, thanks to the high utilization powered by flexible dataflow, and low-cost SFU and voting, VEDA attains the smallest area cost, and the highest energy efficiency (it remains true after technology scaling [13]). Compared with the SOTA edge GPU, VEDA achieves 38.8 $\times$ energy efficiency. One VEDA has a throughput of 18.6 tokens/s. Compensating for the resource discrepancy, 8-VEDA will have a 2.86 $\times$ throughput improvement over GPU. The evaluations demonstrate the feasibility of LLMs edge deployment through VEDA.

TABLE II
COMPARISON WITH RELATED ACCELERATORS

Accelerator Comparison	Sanger	Spatten	VEDA
Support	Attention	Transformer	LLM
Technology[nm]	55	40	28
Area[mm^2]	16.9	1.55	1.06
Throughput[$GOPS$]	529	360	245
Energy Efficiency[$GOPS/W$]	192	382	653
End-to-end Comparison with NVIDIA 4090 GPU			
Average Energy Efficiency: 38.8 $\times$ (Core+Off-chip HBM)			
Average Throughput: 1-VEDA: 18.6 tokens/s, 8-VEDA: 2.86 $\times$ over GPU			

VII. CONCLUSION

In conclusion, this work addresses the computational challenges of LLM inference on edge devices through *voting-based* KV cache eviction, *flexible-product* dataflow, *element-serial scheduling*, a pioneering algorithm-hardware-dataflow tri-optimizations. The resulting accelerator, VEDA, demonstrates significant performance improvements, enabling efficient LLM inference on resource-constrained platforms, and promoting real-time processing and data privacy.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] M. Adnan, A. Arunkumar, G. Jain, P. Nair, I. Soloveychik, and P. Kamath, “Keyformer: Kv cache reduction through key tokens selection for efficient generative inference,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.
- [3] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [6] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee *et al.*, “A³: Accelerating attention mechanisms in neural networks with approximation,” in *2020 IEEE International Symposium on High Performance Computer Architecture*. IEEE, 2020, pp. 328–341.
- [7] Y. Kim, W. Yang, and O. Mutlu, “Ramulator: A fast and extensible dram simulator,” *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [8] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, “Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [9] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, “Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.
- [10] M. Milakov and N. Gimelshein, “Online normalizer calculation for softmax,” *arXiv preprint arXiv:1805.02867*, 2018.
- [11] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, “Cacti 6.0: A tool to model large caches,” *HP laboratories*, vol. 27, p. 28, 2009.
- [12] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, “Compressive transformers for long-range sequence modelling,” *arXiv preprint arXiv:1911.05507*, 2019.
- [13] S. Sarangi and B. Baas, “Deepscaletool: A tool for the accurate estimation of technology scaling in the deep-submicron era,” in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021, pp. 1–5.
- [14] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, “Softmax: Hardware/software co-design of an efficient softmax for transformers,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 469–474.
- [15] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [16] H. Wang, Z. Zhang, and S. Han, “Spatten: Efficient sparse attention architecture with cascade token and head pruning,” in *2021 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 2021, pp. 97–110.
- [17] Z. Wang, G. Wang, H. Jiang, N. Xu, and G. He, “Cosa: Co-operative systolic arrays for multi-head attention mechanism in neural network using hybrid data reuse and fusion methodologies,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [18] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv preprint arXiv:2309.17453*, 2023.
- [19] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for {Transformer-Based} generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [20] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, “Nn-lut: Neural approximation of non-linear operations for efficient transformer inference,” 2021. [Online]. Available: <https://arxiv.org/abs/2112.02191>
- [21] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.